

Dal prompt al file sorgente

Dario Pescini

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

`dario.pescini@unimib.it`

Come si programma in Python

Esistono tre forme di interazione con l'interprete:

- **da linea di comando** utile per testare porzioni di codice o come calcolatrice avanzata
- **con codice sorgente** utile per programmi strutturati
- **da notebook** utile per analisi esplorative dei dati o prototipizzazione

Come si programma in Python

- da linea di comando
 - 1 avvio dell'interprete dei comandi `python`
 - 2 introduzione dell'istruzione
 - 3 output
 - 4 introduzione dell'istruzione
 - 5 output
 - 6 ...

Come si programma in Python

- con codice sorgente

- 1 scrittura del file sorgente

Creazione, con un editor di testo, del file `sorgente.py` che contiene le istruzioni nel linguaggio di programmazione Python.

- 2 esecuzione del programma

Il calcolatore interpreta ed esegue la sequenza di istruzioni codificate all'interno del programma.

```
python sorgente.py
```

da linea di comando: python

Prompt >>> dove inserire il comando

```
dario@vulcano: python
Python 2.7.10 (default, May 25 2015, 13:06:17)
[GCC 4.2.1 Compatible Apple LLVM 6.0 (clang-600.0.56)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> latoA=3
>>> latoB=5
>>> area=latoA*latoB
>>> print area
15
>>> _
```

per uscire dal prompt digitare: `exit()` oppure `ctrl + D`

da linea di comando: ipython

Prompt `In[ ]`: dove inserire il comando

```
dario@vulcano: ipython
Python 2.7.10 (default, Oct 5 2015, 09:41:46)
Type "copyright", "credits" or "license" for more information.

IPython 4.0.0 -- An enhanced Interactive Python.
?                -> Introduction and overview of IPython's features.
%quickref        -> Quick reference.
help             -> Python's own help system.
object?         -> Details about 'object', use 'object??' for extra details.

In [1]: latoA = 3

In [2]: latoB = 5

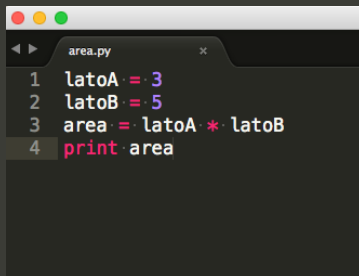
In [3]: area = latoA * latoB

In [4]: print area
15

In [5]: _
```

per uscire dal prompt digitare: `exit()` oppure `ctrl + D`

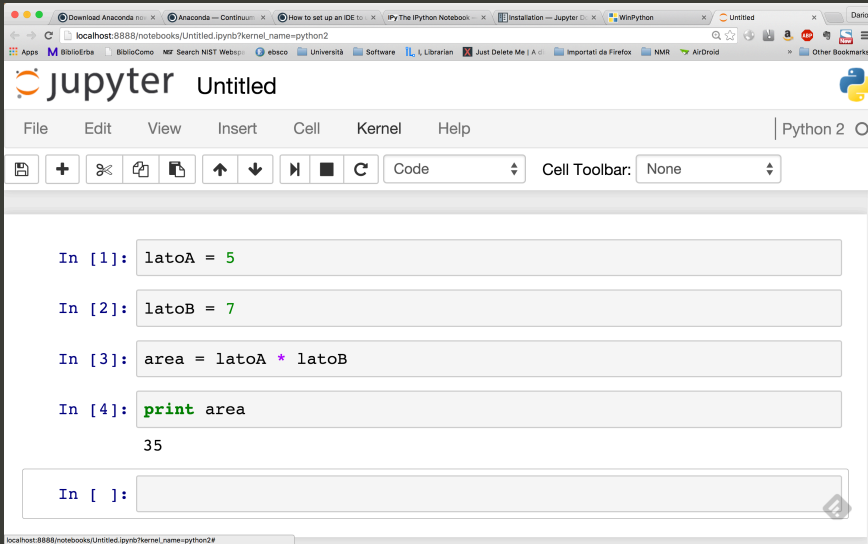
file sorgente



```
1 latoA = 3
2 latoB = 5
3 area = latoA * latoB
4 print area
```

```
darío@vulcano: python area.py
15
darío@vulcano: _
```

notebook



The screenshot displays a Jupyter Notebook interface in a web browser. The browser's address bar shows the URL `localhost:8888/notebooks/Untitled.ipynb?kernel_name=python2`. The notebook's title bar reads "jupyter Untitled". The menu bar includes "File", "Edit", "View", "Insert", "Cell", "Kernel", and "Help". The "Python 2" kernel is selected. The toolbar contains icons for saving, adding, deleting, and duplicating cells, as well as navigation and execution controls. The "Cell Toolbar" is set to "None". The notebook contains four input cells with the following code:

```
In [1]: latoA = 5
```

```
In [2]: latoB = 7
```

```
In [3]: area = latoA * latoB
```

```
In [4]: print area
```

The output of the fourth cell is `35`. Below the code cells is an empty input cell labeled `In [ ]:`. The status bar at the bottom shows the URL `localhost:8888/notebooks/Untitled.ipynb?kernel_name=python2#`.

Struttura di un file Sorgente

Un file sorgente si suddivide in:

- importazione di librerie
- funzioni
- sequenza di istruzioni

Elementi di un file sorgente

1 commenti: `# commento`

servono a rendere più leggibile il codice per un suo riutilizzo.

2 istruzioni Python: `istruzione`

```
istruzione1  
istruzione2
```

```
...
```

sono i “comandi” veri e propri; possono essere separate in gruppi logici per mezzo di un linea vuota.

3 indentazioni: `...4 spazi o multipli`

```
istruzione1  
...istruzione2  
...istruzione3  
istruzione4
```

servono a creare blocchi di istruzioni per il controllo del flusso dell'informazione.

Elementi di un file sorgente

```
percorso.py
1 import math
2
3
4 # questo programma calcola la
5 # lunghezza del percorso che unisce
6 # più città.
7 # Viene terminato tramite l'inserimento di
8 # una coordinata negativa.
9
10 haltFlag = 1 # inizializzo flag
11 lunghezza = 0 #inizializzo a 0 per successiva sommatoria
12
13 print "inserire le coordinate delle città:"
14 print "un valore negativo termina l'inserimento"
15 x0ld = input("ascissa ")
16 y0ld = input("ordinata ")
17 while (haltFlag > 0):
18     x = input("ascissa ")
19     y = input("ordinata ")
20     if(x < 0 or y < 0):
21         haltFlag = 0
22     else :
23         deltaX = x - x0ld
24         deltaY = y - y0ld
25         lunghezza += (deltaX**2 + deltaY**2)
26 lunghezza = math.sqrt(lunghezza)
27 print "la lunghezza del percorso e' ", lunghezza
28
```

importazione modulo

commento
su più
linee

indentazione
4 spazi

blocco di
istruzioni

Attenzione!

- **Case Sensitive** `pippo`, `Pippo`, `Pipp0`, `PiPp0` sono diversi in Python.
- **Parole riservate** Esiste una insieme di identificatori propri del linguaggio che non possono essere ridefiniti:

<code>False</code>	<code>None</code>	<code>True</code>	<code>and</code>	<code>as</code>
<code>assert</code>	<code>async</code>	<code>await</code>	<code>break</code>	<code>class</code>
<code>continue</code>	<code>def</code>	<code>del</code>	<code>elif</code>	<code>else</code>
<code>except</code>	<code>finally</code>	<code>for</code>	<code>from</code>	<code>global</code>
<code>if</code>	<code>import</code>	<code>in</code>	<code>is</code>	<code>lambda</code>
<code>nonlocal</code>	<code>not</code>	<code>or</code>	<code>pass</code>	<code>raise</code>
<code>return</code>	<code>try</code>	<code>while</code>	<code>with</code>	<code>yield</code>

Primo esempio di istruzioni

- `print(variable)` stampa a video una variabile

scrittura

```
print("HelloWorld!")
```

- `input('Stringa')` stampa Stringa a video e **legge da tastiera** quanto viene digitato come una stringa

lettura di una variabile a

```
a = input("inserire il contenuto della variabile a " )
```

Primo esempio di istruzioni

- `eval('12.0')` cerca di convertire la stringa '12.0' in valore numerico corrispondente

conversione di una stringa in un numero

```
numero = eval(stringa)
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Esempio I/O

```
>>> a = 100
>>> print(a)
100
```

```
>>> b = input('inserisci un valore ')
inserisci un valore 100
>>> print(eval(b))
100
>>> c = input('inserisci un valore ')
inserisci un valore 'deiCaratteri'
>>> print(c)
deiCaratteri
```

Dal prompt al sorgente

In uno script sarebbe:

A screenshot of a code editor window titled 'dalPromptAlSorgente.py'. The editor shows a Python script with the following code:

```
1 a = 100~
2 print(a)~
3 b = input('inserisci un valore ') # richiesta inserimento primo valore~
4 print (b)~
5 c = input('inserisci un valore ') # richiesta inserimento secondo valore~
6 print (c)~
7
```

The code is displayed on a dark background with syntax highlighting: numbers are light blue, variable names are white, and string literals are green. The cursor is at the end of line 7.

Dal prompt al sorgente

In uno script sarebbe:



```
dalPromptAlSorgente.py
1 a = 100
2 print(a)
3 b = input('inserisci un valore ') # richiesta inserimento primo valore
4 print (b)
5 c = input('inserisci un valore ') # richiesta inserimento secondo valore
6 print (c)
7
```

```
dario@vulcano: ipython dalPromptAlSorgente.py
100
inserisci un valore 100
100
inserisci un valore 'deiCaratteri'
deiCaratteri
dario@vulcano: _
```